

strategynet.ai

Slice + Hypercube II

Stateful factor circuits, explicit backpressure, and deterministic replay

PART II: IMPLEMENTATION AND EVIDENCE • VERSION 0.1

research@strategynet.ai

Open-source working paper • July 2026

ABSTRACT

The first Slice + Hypercube paper established a live state plane and a pure, generation-oriented calculation graph. It deliberately left stateful function cells, durable replay, transport, and effects as future boundaries. This companion paper implements a first stateful callback boundary and a deterministic recording/replay path.

hypercube-circuit accepts one coherent Hypercube snapshot at a time, orders it through a single-producer Disruptor ring, and invokes deterministic stateful processors using generation time rather than processor wall time. The reference processor turns a calculated cross-section into persistent threshold state with hysteresis and missing-data invalidation. Every generation emits both a complete state cross-section for late subscribers and sparse transition events for synchronized consumers.

The same path records a versioned manifest, complete Hypercube updates, optional upstream stream positions, semantic snapshot digests, state, and transitions. Replay constructs a fresh engine and fresh processor state, changes only the execution mode, and compares every generation. A checked-in demonstration records 40 generations over 32 entities, recalculates 6,400 factor cells, reconstructs 1,280 trigger-state cells and 78 transitions, and reports zero divergent generations. The ETF, pairs, browser, and replay programs provide an executable application ladder. The paper does not claim a production Aeron adapter, raw-feed reconstruction, checkpoint seeking, latency benchmark, or trading authority; it specifies those boundaries explicitly.

Contents

I From Factors to Temporal State	3	15 Effects remain outside replay	10
1 Continuation from Part I	3	IV Transport and Deployment	11
2 Application ladder	3	16 Local circuit versus Aeron	11
3 Three different kinds of truth	4	17 Source positions define the input frontier	11
II Circuit Contract	5	18 Deployment patterns	11
4 One coherent generation is one event	5	V Executable Demonstrations	12
5 Why a Disruptor is useful here	5	19 ETF: two axes remain visible	12
6 Backpressure is an application policy	6	20 Pairs: model residual becomes signal state	13
7 Processor determinism	6	21 Browser: one generation, several views	13
8 Reference threshold state machine	6	22 Record/replay experiment	14
9 Dense state and sparse transitions	7	23 Failure contracts exercised by tests	14
III Deterministic Record and Replay	8	24 Current conformance result	14
10 Choosing the replay boundary	8	VI Application and Research Sequence	15
11 Versioned recording	8	25 The business application	15
12 Semantic exactness	8	26 Counterfactual replay	15
13 Replay algorithm	9	27 Implemented limits	15
14 Divergence is a result, not an exception	9	28 Next implementation sequence	16
		29 Conclusion	16

PART I

From Factors to Temporal State

1 Continuation from Part I

Part I separated three responsibilities [1]: Slice publishes typed current vectors, Hypercube evaluates a coherent generation through a declared graph, and external adapters move or act on the resulting state. That release could calculate a factor, rank, basket discrepancy, or cointegrating residual. It could not remember that a condition had persisted for three generations, distinguish an entry from an already-active signal, or reconstruct that temporal state from a recording.

Those omissions become visible in an ordinary factor application. Let $f_{i,g}$ and $h_{i,g}$ be two calculated factors for entity i at generation g . A linear Hypercube node can calculate a spread

$$d_{i,g} = f_{i,g} - h_{i,g}.$$

A trading, alerting, or monitoring application rarely wants a new action on every update for which $d_{i,g} > a$. It usually wants a lifecycle:

inactive $\rightarrow$ qualifying $\rightarrow$ active $\rightarrow$ exited or invalidated.

That lifecycle is state across generations, not another cross-sectional normalization within one generation.

Core thesis

The useful product is not merely a stream of factor values. It is a reconstructible state transition system whose inputs, logical clock, current state, sparse changes, overload behavior, and effect boundary are all explicit.

2 Application ladder

The public repository now contains four complementary demonstrations [8]:

Demonstration	Implemented operation	Architectural evidence
browser server	factor graph over a changing stock cross-section; Slice, JSON, and SSE publication	one calculation can support local memory and remote presentation views
ETF monitor	constituent-axis basket reduction followed by ETF-axis ranking	domain arithmetic and cross-sectional algebra remain separate, visible stages
pairs monitor	cointegrating residual, model standardization, half-life, and opportunity rank	model values remain aligned to stable pair entities across generations
record/replay	factor calculation, persistent trigger state, versioned recording, and fresh reconstruction	the temporal layer can be verified without invoking a live effect

The demonstrations are deliberately not collapsed into one large application. ETF and pairs make the upstream calculations readable. The browser makes publication visible. The circuit demonstration isolates temporal state and replay. Their shared contract is a coherent Hypercube generation.

3 Three different kinds of truth

The architecture distinguishes:

1. **current analytical truth**: the latest coherent values and callback state a consumer may read;
2. **historical reconstruction truth**: the ordered inputs and metadata required to rebuild those values and state;
3. **effect authority**: the separate capability to publish, persist, alert, or trade.

A ring buffer is not durable history. A memory-mapped vector is not an event log. A replayable decision is not permission to execute it. Keeping these distinctions structural avoids two common errors: treating a fast queue as the system of record, and allowing deterministic calculation code to inherit irreversible authority.

PART II

Circuit Contract

4 One coherent generation is one event

For generation g , the pure engine receives

$$U_g = (g, t_g, m_g, R_g, N_g)$$

and emits a coherent snapshot

$$H_g = E(U_g) = (g, t_g, m_g, n_g, V_g, Q_g).$$

Here R_g is the complete entity-row cross-section, N_g is the declared node graph, V_g is the ordered calculated value set, and Q_g is node status. The circuit does not subscribe to individual cell writes. Its event is the immutable H_g , after the engine has accepted the complete update.

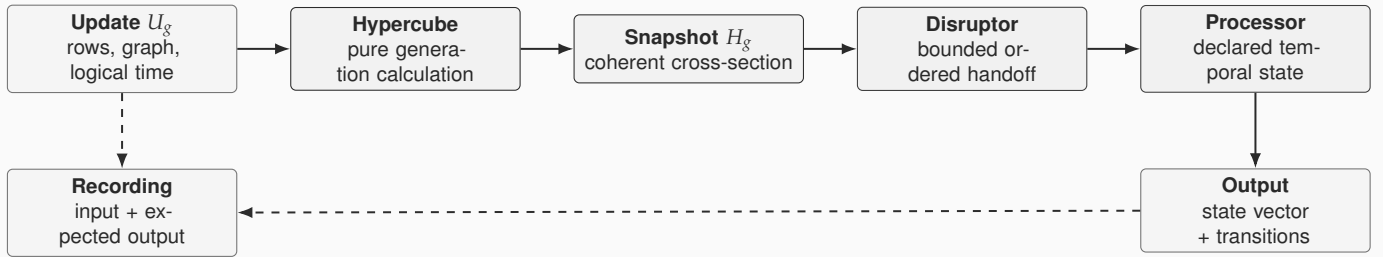


Figure 1. Implemented calculation, circuit, and recording boundary. A callback never observes a half-built Hypercube generation.

The per-frame context is

$$K_g = (q_g, g, t_g, m_g),$$

where q_g is the circuit sequence. Stateful code uses g and t_g as logical coordinates. Processor scheduling time, operating-system time, and measured calculation duration are operational observations, not inputs to the state transition function.

5 Why a Disruptor is useful here

The LMAX Disruptor separates event storage, sequencing, producer claims, consumer progress, and dependency graphs around a preallocated bounded ring [2]. The Rust `disruptor` crate implements the same family of patterns for low-latency inter-thread communication and recommends nonblocking publication when the application must handle a full ring explicitly [3].

The current circuit chooses a single producer because Hypercube generations already have one ordered owner. One stateful processor consumes them in sequence. The ring offers:

- a preallocated bounded handoff rather than unbounded queue growth;
- a circuit sequence independent of wall-clock timestamps;
- nonblocking submission with a visible `RingFull` result;
- configurable sleep, spin-loop, and busy-spin waiting;
- an asynchronous submit/receive API and a lockstep correctness path.

It does *not* make an overloaded consumer current. If producer sequence p , consumer frontier c , and ring capacity B satisfy

$$p - c \geq B,$$

the next nonblocking publication is rejected. That failure is valuable information: the consumer cannot keep up and latency would otherwise grow behind the producer.

6 Backpressure is an application policy

The circuit never silently overwrites an unprocessed correctness-lane generation. On a full ring, an owner must choose one of three policies:

1. backpressure the producer and preserve every accepted generation;
2. retain normalized input durably and catch up from its log position;
3. coalesce or drop only a separately named best-effort view.

CaptureSession and ReplayRunner use lockstep processing. The engine does not accept the next update until callback processing and the recording step complete. Where an upstream market feed cannot be slowed, the feed should be archived before generation assembly. The circuit is then a deterministic consumer of durable input, not the sole owner of history.

What the ring solves

The ring makes ordering, capacity, and consumer progress visible inside one process. It does not solve raw-feed durability, cross-host recovery, multi-stream generation assembly, or external-effect idempotency.

7 Processor determinism

A processor is a state transition function

$$(S_g, O_g) = P(S_{g-1}, K_g, H_g),$$

where S_g is private declared state and O_g is the published deterministic output. Replay is meaningful when:

1. P reads only S_{g-1} , K_g , H_g , and immutable configuration;
2. iteration and output ordering are canonical;
3. missing data and reset behavior are specified;
4. nondeterministic I/O, random sources, and wall time stay outside P ;
5. irreversible effects consume O_g through another adapter.

Logical ordering has a longer history than this implementation. Lamport showed that event ordering can be represented independently of synchronized physical clocks [4]. Hypercube does not implement a distributed Lamport clock, but it applies the relevant separation: generation and circuit sequence determine processing order, while observation time describes the input.

8 Reference threshold state machine

The first processor watches one calculated node for every entity. A trigger specification is

$$T = (\text{id}, \text{node}, a, b, p),$$

with entry threshold a , exit threshold b , required persistence p , and strict hysteresis $a > b$.

For an inactive entity with observed value $x_{i,g}$, the qualifying count is

$$k_{i,g} = \begin{cases} k_{i,g-1} + 1, & x_{i,g} \geq a, \\ 0, & x_{i,g} < a. \end{cases}$$

The state becomes active when $k_{i,g} \geq p$. An active state remains active through the hysteresis band and exits only when

$$x_{i,g} \leq b.$$

If the watched node has no value for an active entity, the state is *invalidated*, not silently exited. If a qualifying but inactive entity is missing, its count is removed, so a later value cannot falsely complete a “consecutive” run across a gap.

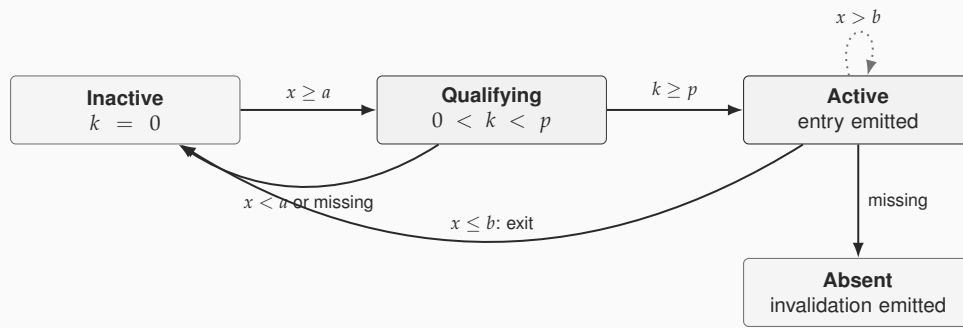


Figure 2. Persistent upper-threshold trigger with hysteresis and explicit missing-data invalidation.

9 Dense state and sparse transitions

For every generation, the processor emits:

$$Z_g = \{(\text{trigger}, \text{node}, i, \text{active}_{i,g}, k_{i,g})\}$$

and

$$\Delta Z_g = \{\text{Entered, Exited, Invalidated events at } g\}.$$

Z_g is the complete ordered trigger-state cross-section. It maps naturally to a new **Slice** vector and lets a late subscriber bootstrap without replaying the entire run. ΔZ_g is the sparse edge stream for a subscriber that already holds Z_{g-1} . Recording both views also exposes latent state divergence even when no transition happens in a generation.

This dual output is the bridge from an Excel-like continuously recalculated factor sheet to an application state machine. Factor values remain inspectable, signal lifecycle becomes explicit, and downstream consumers can choose snapshot or event semantics without recalculating either.

PART III

Deterministic Record and Replay

10 Choosing the replay boundary

“Replay the system” is incomplete until the recorded boundary is named.

Recorded boundary	What replay reconstructs	Tradeoff
published factor frame	callbacks, strategies, and downstream state only	smallest record; cannot verify factor calculation
complete Hypercube update	factor graph, values, callback state, and transitions	implemented compromise; repeats rows and graph per generation
normalized raw events plus generation seals	ingestion, generation assembly, factors, and callbacks	strongest apparatus replay; requires ordering, watermark, and join contracts

The current implementation records complete Hypercube updates. It is more useful than factor-only replay because changes to rows, node graphs, transforms, or arithmetic can be detected. It stops short of claiming that vendor normalization and multi-feed generation assembly are reproducible.

11 Versioned recording

A recording is

$$\mathcal{D} = (M, G_0, G_1, \dots, G_n).$$

The manifest

$$M = (v, \text{run}, \text{build}, \text{config}, \text{layout}, \text{triggers}, \text{metadata})$$

identifies the schema and non-generation configuration. One generation record is

$$G_g = (q_g, U_g, F_g, D_g, Z_g, \Delta Z_g),$$

where F_g is an optional map from stable upstream stream identity to the last position included in the generation and D_g is the expected semantic snapshot digest.

The development adapter writes one manifest and then one generation per JSON line. It validates:

- exactly one leading version-1 manifest;
- contiguous circuit sequences and increasing generations;
- agreement between update and expected output generation;
- valid trigger configuration and nonempty provenance fields;
- only finite JSON numeric inputs.

JSON parsing enables exact finite $f64$ round trips. NaN and infinity are rejected before the engine advances because JSON would otherwise encode them as `null`; a missing primitive is represented by omitting its field. The format is intentionally inspectable and not presented as the final low-latency wire encoding.

12 Semantic exactness

The semantic snapshot digest includes:

- generation, observation time, and entity count;
- ordered node and entity identities;
- exact IEEE-754 value bits and value observation times;
- ordered status identities and value/missing counts.

It excludes:

- ExecutionMode, because Live becomes Replay;
- measured node compute duration;
- wall time, thread identity, transport delay, and logging details.

The current digest is a versioned streaming xxHash64 over a canonical byte representation. Digest equality is supplemented by exact comparison of the full ordered state and transition outputs. The build and configuration identities remain provenance: a different compiler, target, or implementation may be replayed, but any arithmetic difference is reported rather than normalized away.

Exact does not mean identical runtime

Replay requires semantic equality of calculated values and state transitions. It does not require identical processor scheduling, compute duration, or transport timing. Including those operational measurements would make a correct replay fail for irrelevant reasons.

13 Replay algorithm

Replay performs the following bounded procedure:

1. validate and read the recording manifest;
2. construct a new HypercubeEngine;
3. construct a new processor from the recorded trigger configuration;
4. set each recorded update mode to Replay;
5. calculate the snapshot and process it in lockstep;
6. compare circuit sequence, semantic digest, complete state, and transitions;
7. count every divergent generation and retain the first concrete expected-versus-actual difference.

Formally,

$$H'_g = E'(U_g[m \leftarrow \text{Replay}]),$$

$$(S'_g, O'_g) = P'(S'_{g-1}, K'_g, H'_g).$$

An exact replay requires

$$q'_g = q_g, \quad D(H'_g) = D_g, \quad Z'_g = Z_g, \quad \Delta Z'_g = \Delta Z_g$$

for every recorded generation.

Research on deterministic component replay likewise separates virtual or logical ordering from incidental runtime scheduling [6]. The Hypercube implementation is narrower: it records already coherent application updates and executes one ordered stateful lane.

14 Divergence is a result, not an exception

Malformed files, invalid updates, and processor failures are operational errors. A valid recording that produces a different digest, state, or transition is a semantic result. The CLI therefore uses separate exits:

Exit	Meaning
0	every recorded generation matched
1	operational or recording-format failure
2	replay completed with semantic divergence

This distinction supports two workflows:

- **reconstruction**: expect zero divergence under the same implementation contract;
- **counterfactual replay**: intentionally change a node, threshold, or model and analyze the resulting divergence.

15 Effects remain outside replay

Event-sourced systems must prevent replayed events from reissuing external updates to systems that cannot distinguish replay from live processing [5]. Hypercube enforces the simpler precursor: the MVP contains no execution adapter, and processors are documented as deterministic calculation code without irreversible effects.

Future replay publication should use a namespace such as

```
replay/{run_id}/. . .
```

and every authorized effect adapter must reject replay-mode inputs. A live order, notification, or database mutation should also carry an idempotency key and an evidence reference. Calculation replay and effect replay are separate capabilities.

PART IV

Transport and Deployment

16 Local circuit versus Aeron

Inside one compute process, Aeron is unnecessary for the engine-to-callback handoff. The immutable snapshot is already in memory; the Disruptor provides ordered inter-thread transfer without serialization or a media driver.

Aeron becomes useful at a process or host boundary:

- factor/state publication to independent services;
- durable capture of normalized inputs or generation envelopes;
- subscriber recovery from a known stream position;
- controlled replay onto a dedicated channel.

Aeron Archive records and replays streams from durable storage and supports replay from a specified recording and position [7]. The current crate does not depend on Aeron. Its public manifest and generation structs are transport-neutral envelopes that a future SBE adapter can encode as:

```
hypercube.recording.manifest.v1
hypercube.recording.generation.v1
```

17 Source positions define the input frontier

For k upstream streams, generation g may record

$$F_g = (p_{1,g}, p_{2,g}, \dots, p_{k,g}),$$

where $p_{j,g}$ is the last input position included from stream j . The manifest can carry Archive recording identities and start positions; `source_positions` carries the per-generation frontier.

This vector makes a generation's input cut inspectable, but it does not by itself prove that the cut is semantically coherent. The application must still define:

- the canonical order when inputs from different streams race;
- event-time and ingestion-time watermark rules;
- late, duplicate, corrected, and missing event behavior;
- the generation seal that says no more input belongs to g .

Those rules are the remaining step from complete-update replay to raw-feed apparatus replay.

18 Deployment patterns

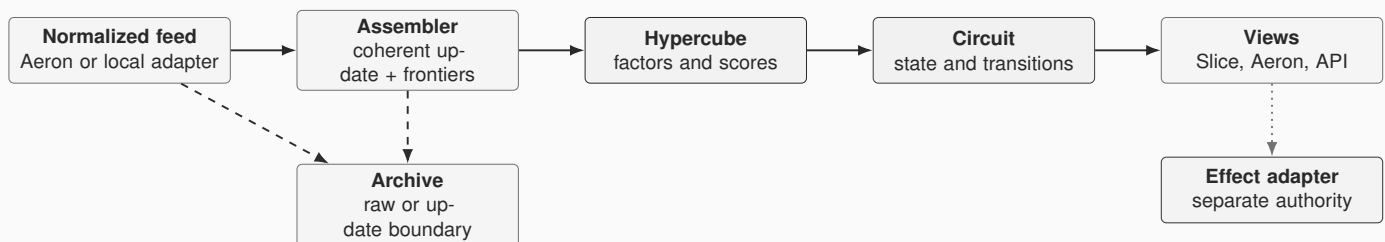


Figure 3. Target deployment: live calculation (solid), recording (dashed), and separately authorized effects (dotted).

The topology supports a **single process**; a **split deployment** that keeps engine-to-circuit transfer local but publishes views over Aeron; or **durable reconstruction** that archives normalized input and publishes replay through an isolated namespace.

PART V

Executable Demonstrations

19 ETF: two axes remain visible

The ETF monitor reduces 160 correlated constituent returns into 12 basket fair values and then evaluates five ETF-aligned Hypercube nodes. For ETF j ,

$$r_{j,g}^{\text{NAV}} = \sum_i w_{ji} r_{i,g}, \quad V_{j,g} = V_{j,g-1}(1 + r_{j,g}^{\text{NAV}}),$$

while the simulated market premium follows

$$u_{j,g} = 0.88u_{j,g-1} + 0.00040\eta_{j,g}.$$

The graph publishes fair value, market price, premium basis points, model z-score, and cross-sectional premium rank. The terminal therefore displays 60 cells per generation.

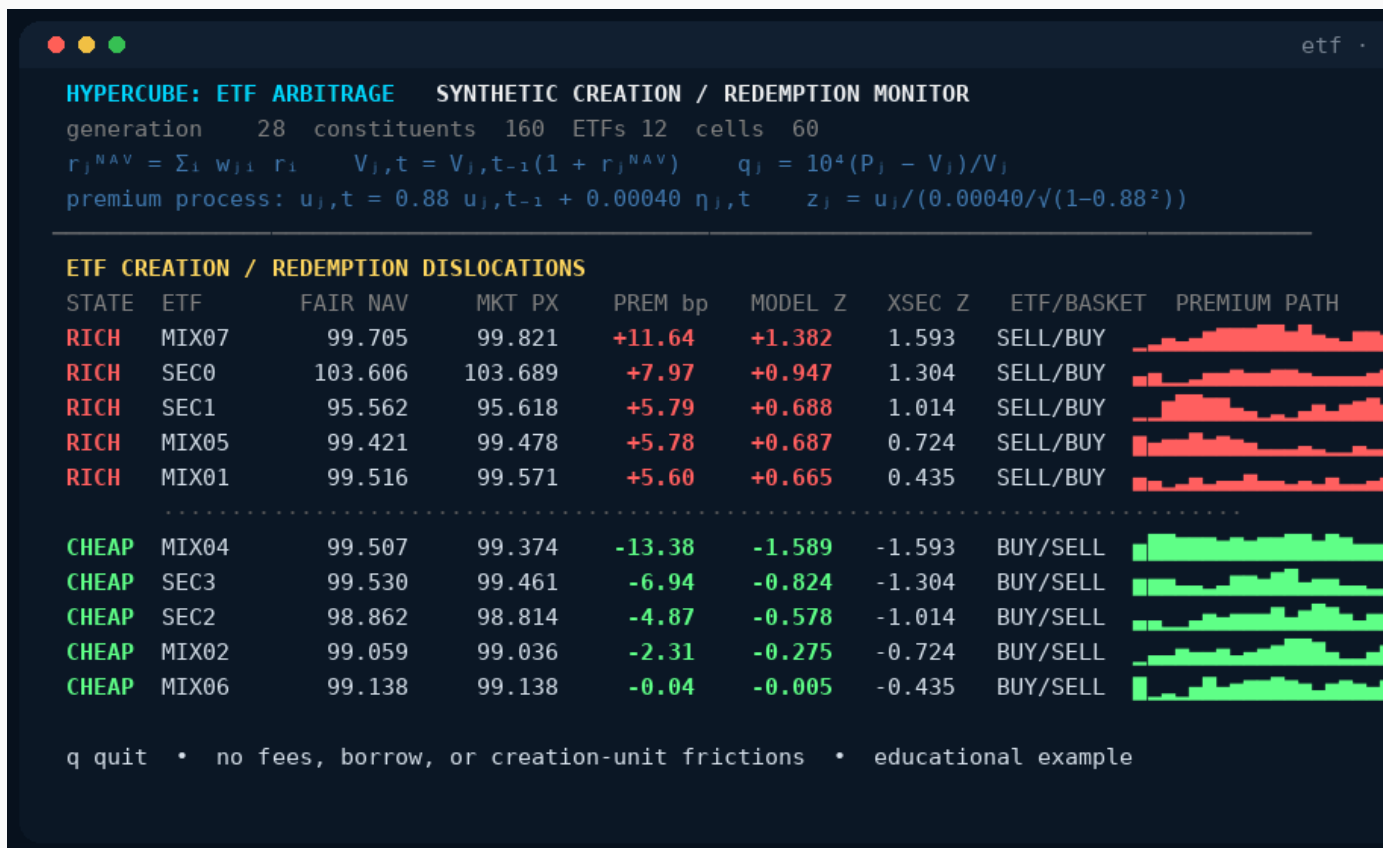


Figure 4. Checked-in ETF recording at generation 28. Labels preserve the premium sign independently of color. Fees, borrow, and creation-unit frictions remain explicit omissions.

This is an upstream circuit application: a callback may watch `premium_z`, require persistence, and publish entry/exit state without changing the basket calculation. A more complete application would combine factor state with inventory, executable spread, creation size, and policy before producing a bounded intent.

20 Pairs: model residual becomes signal state

The pairs monitor publishes 120 cells over 24 stable pair entities. Its residual and standardization are

$$y_{j,g} = \log A_{j,g} - \alpha_j - \beta_j \log B_{j,g}, \quad z_{j,g} = \frac{y_{j,g}}{\sigma_j / \sqrt{1 - \phi_j^2}}.$$

Half-life and cross-sectional opportunity rank remain separate nodes.

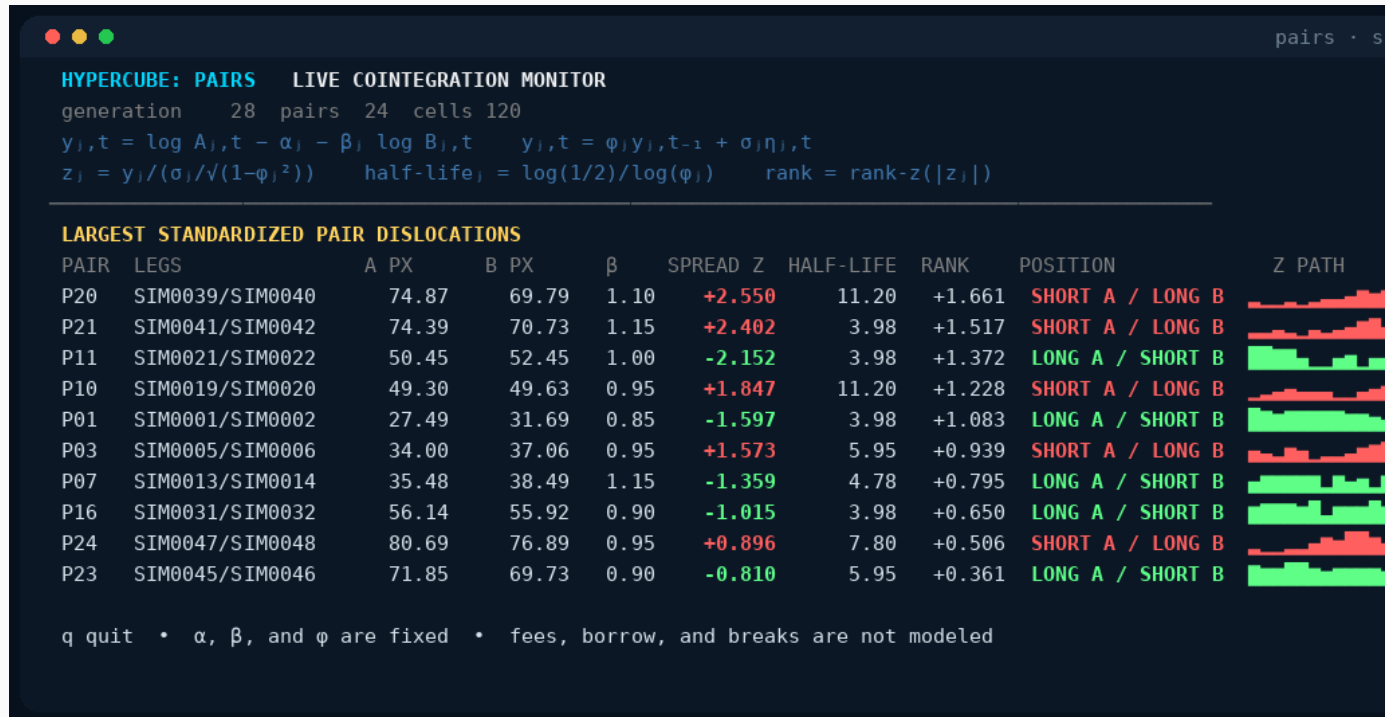


Figure 5. Checked-in pairs recording at generation 28. The display ranks standardized dislocations while retaining hedge ratio and half-life context.

A threshold processor turns a repeated z-score condition into lifecycle state. Hysteresis avoids an exit/re-entry loop near the entry boundary; invalidation distinguishes a missing leg or suppressed residual from a genuine convergence exit. Estimation windows, structural-break detection, borrow, cost, and portfolio coupling remain outside the demo.

21 Browser: one generation, several views

The synthetic server computes five nodes for a changing stock cross-section, publishes them to entity-aligned Slice files, exposes a JSON snapshot, and streams generations through SSE to a dependency-free dashboard. It demonstrates that current-state projection and subscriber presentation need not rerun factor arithmetic.

The circuit extends that fan-out with two additional views:

- a dense current-state projection suitable for Slice or a bootstrap response;
- a sparse transition stream suitable for already synchronized subscribers.

Neither browser SSE nor Slice is the replay log. They are projections of a generation whose reproducible input lives at the recording boundary.

22 Record/replay experiment

The executable experiment is:

```
cargo run -p hypercube-circuit --bin hypercube-replay -- \
  record-demo /tmp/hypercube-factor.jsonl 40 32

cargo run -p hypercube-circuit --bin hypercube-replay -- \
  verify /tmp/hypercube-factor.jsonl
```

The synthetic graph has five nodes over 32 entities for 40 generations:

$$5 \cdot 32 \cdot 40 = 6,400$$

factor cells. One threshold specification yields

$$32 \cdot 40 = 1,280$$

complete trigger-state cells.

Compared object	Recorded	Replayed
generations	40	40
factor cells represented by semantic digests	6,400	6,400
trigger-state cells	1,280	1,280
transitions	78	78
divergent generations		0

The file contains 41 lines: one manifest and 40 generation records. This is a functional exactness result, not a throughput or storage benchmark.

23 Failure contracts exercised by tests

The circuit tests do more than assert the happy path:

- a blocked consumer fills a two-slot ring and the third generation returns `RingFull` without overwriting pending work;
- a missing inactive candidate resets persistence, while a missing active entity emits `Invalidated`;
- sequence gaps and non-increasing generations are rejected;
- non-finite JSON input is rejected before a generation is written;
- live and replay JSON values reproduce exact floating-point bits;
- a changed recorded primitive produces a snapshot-digest divergence;
- unchanged input reconstructs snapshot, state, and transitions.

These tests target the places where a superficially working event system can quietly drift: overload, gaps, missing observations, serialization, hidden state, and nondeterministic comparison.

24 Current conformance result

The workspace currently runs 29 tests:

Suite	Tests	Result
engine, publisher, and synthetic injectors	6	passed
ETF and pairs example invariants	2	passed
Slice layout, mmap, records, and vector algebra	7	passed
circuit, triggers, recording, digest, and replay	14	passed
Total	29	passed

The workspace also passes formatting, clippy with warnings denied, and public API documentation with warnings denied. These are conformance and packaging checks; the repository still makes no latency or throughput claim.

PART VI

Application and Research Sequence

25 The business application

The immediate application is a factor-to-signal state service:

1. Hypercube calculates raw fields, factor spreads, ranks, residuals, and scores over the full current universe;
2. the circuit applies persistence, hysteresis, invalidation, and later stateful model callbacks;
3. dense state is published for current consumers;
4. sparse transitions feed alerts, strategy coordinators, portfolio construction, or a separately governed trading adapter;
5. the recording reconstructs why a state existed and when it changed.

The commercial value is less about inventing another formula and more about making calculated state reusable. A factor team owns one factor definition; multiple applications can consume the same current vector and transition history. A strategy can be tested against the exact inputs that drove the live system. An operator can distinguish “no longer qualifies” from “source disappeared.” A new subscriber can bootstrap from dense state instead of inferring it from an incomplete recent event window.

26 Counterfactual replay

Once reconstruction is exact, replay can branch:

- change entry, exit, or persistence parameters;
- replace one factor node or model version;
- compare old and new state transitions by generation;
- inject delayed or missing data according to a declared scenario;
- measure proposed decision intents without enabling live effects.

This resembles the “parallel model” enabled by event sourcing: alternative state can be produced from an alternative event or processing history [5]. The current CLI verifies only the recorded configuration. A counterfactual runner and comparative report are a natural next layer.

27 Implemented limits

The current release establishes a vertical circuit, not a complete event platform:

- JSON Lines is the only recording adapter; Aeron Archive and SBE mapping are specified but not implemented;
- complete updates are replayed, not raw vendor events or generation assembly;
- one threshold processor is provided; general callback composition, window state, and model artifacts need additional schemas;
- replay begins at generation zero; state checkpoints and seek are not implemented;
- xxHash64 detects ordinary divergence but is not a cryptographic authenticity proof;
- state vectors are public structs but no dedicated trigger-to-Slice publisher is included;
- no benchmark justifies a wait strategy, ring size, or deployment topology for production;
- no external-effect or trading authority exists in the circuit crate.

These limits are intentional. They keep a replay verification command from being confused with a production recovery plan or a calculated signal from being confused with an executable order.

28 Next implementation sequence

A bounded continuation is:

1. add an SBE codec for manifest and generation envelopes;
2. record and replay those envelopes through Aeron Archive by identity and position;
3. define canonical multi-stream generation seals and late-event policy;
4. add periodic processor checkpoints with digest-verified seek;
5. publish trigger state into Slice and transitions onto a named subscriber channel;
6. add counterfactual configuration and first-divergence comparison;
7. benchmark declared topologies with percentile latency, backlog, CPU, allocation, and recovery-time measurements;
8. specify an idempotent simulated effect adapter before any live execution integration.

29 Conclusion

Part I made current analytical values cheap to share and deterministic to recalculate within one generation. Part II adds the missing temporal boundary. A coherent snapshot becomes one ordered circuit event; a declared processor turns it into reconstructible state; a bounded ring exposes overload instead of hiding it; and a versioned recording proves whether a fresh engine and processor arrive at the same result.

The architecture now supports a useful end-to-end statement:

input generation $\longrightarrow$ factor state $\longrightarrow$ signal state $\longrightarrow$ recorded evidence.

ETF and pairs show readable financial calculations, the browser shows multi-view publication, and the replay demo verifies the stateful layer. Aeron, raw-feed reconstruction, checkpoints, benchmarks, and effects remain separate work, but their contracts are now narrower and testable.

The central design choice is restraint. The Disruptor moves coherent state; it is not truth. The recording reconstructs calculation; it is not authority. The callback emits a decision state; it is not an effect. Those distinctions allow the system to become faster and more composable without making its behavior less explainable.

References

- [1] StrategyNet, *Slice + Hypercube: Current analytical state without rebuilding it for every reader*, Open-source working paper, version 0.2, July 2026, https://github.com/ingvor-restrat/hypercube/blob/main/docs/latex/hypercube_foundations.pdf.
- [2] M. Thompson, D. Farley, M. Barker, P. Gee, and A. Stewart, *LMAX Disruptor: High performance alternative to bounded queues for exchanging data between concurrent threads*, LMAX Exchange, May 2011, <https://lmax-exchange.github.io/disruptor/disruptor.html>.
- [3] *disruptor 4.3.0: Low latency inter-thread communication via a ring buffer*, Rust crate documentation, [https://docs.rs/disruptor/](https://docs.rs/disruptor/4.3.0/disruptor/).
- [4] L. Lamport, *Time, Clocks, and the Ordering of Events in a Distributed System*, Communications of the ACM, 21(7), pp. 558–565, 1978, <https://www.microsoft.com/en-us/research/publication/time-clocks-ordering-events-distributed-system/>.
- [5] M. Fowler, *Event Sourcing*, Further Enterprise Application Architecture, 2005, <https://martinfowler.com/eaDev/EventSourcing.html>.
- [6] R. Strom, C. Dorai, T. H. Feng, and Z. Wei, *Deterministic Replay for Transparent Recovery in Component-Oriented Middleware*, IEEE ICDCS, 2009, <https://doi.org/10.1109/ICDCS.2009.79>.
- [7] Aeron Project, *Aeron Archive Overview*, <https://aeron.io/docs/aeron-archive/overview/>.
- [8] StrategyNet, *Hypercube public repository*, <https://github.com/ingvor-restrat/hypercube>.